*Definitions, Acronyms, and Abbreviations*

| ***Term*** | ***Definition*** |
|---|---|
| **SPAMS** | Student Portfolio Achievement Management System |
| **SRS** | Software Requirements Specification |
| **IEEE** | Institute of Electrical and Electronics Engineers |
| **JWT** | JSON Web Token — a compact, URL-safe token used for authentication |
| **API** | Application Programming Interface |
| **REST** | Representational State Transfer — architectural style for distributed systems |
| **RBAC** | Role-Based Access Control |
| **FR** | Functional Requirement |
| **NFR** | Non-Functional Requirement |
| **CRUD** | Create, Read, Update, Delete operations |
| **ER** | Entity-Relationship |
| **UI** | User Interface |
| **UML** | Unified Modelling Language |
| **Next.js** | React-based web framework used for the SPAMS frontend |
| **NestJS** | Node.js framework used for the SPAMS backend |
| **MongoDB** | Document-oriented NoSQL database used for persistence |
| **Mongoose** | Object Data Modelling (ODM) library for MongoDB and Node.js |
| **Department Head** | A senior faculty member responsible for departmental oversight |
| **Achievement** | An academic or extracurricular accomplishment submitted by a student |
| **Portfolio** | A curated collection of academic work and achievements belonging to a student |
| **Review** | A formal evaluation of a student achievement submitted by a Faculty member |

| | |
|---|---|
| **Audit Log** | An immutable chronological record of system actions for accountability |

# 4. System Features & Requirements

## 4.1 *Student Dashboard*

*4.1.1 Feature Description*

The Student Dashboard is the primary interface for enrolled students. It provides a personalised summary of the student's academic journey including achievement submission, portfolio management, and notification tracking. Access is restricted to users with the student role. *4.1.2 Functional Requirements*

| *ID* | *Requirement* |
|---|---|
| **FR-S-1** | The system shall display a personalised dashboard upon successful student login, showing a summary of submitted achievements grouped by status (pending, under_review, approved, rejected). |
| **FR-S-2** | The system shall allow a student to submit a new achievement by providing: title (max 200 chars), description (max 2000 chars), category (from system-defined list), date, list of skills (comma-separated), and one or more supporting documents (name, URL, type, size). |
| **FR-S-3** | The system shall validate all achievement submission fields server-side; invalid or incomplete submissions shall return HTTP 400 with descriptive error messages per invalid field. |
| **FR-S-4** | The system shall allow a student to view the detail of any previously submitted achievement, including its current status, reviewer name, review comment, and rating if reviewed. |
| **FR-S-5** | The system shall allow a student to create a new portfolio specifying: name, type (academic, projects, certifications, other), and visibility (public / private). |
| **FR-S-6** | The system shall allow a student to add portfolio items to an existing portfolio, each item comprising: title, description, optional duration (start and end dates), and optional attachments. |
| **FR-S-7** | The system shall generate a unique, shareable public link (shareLink) for portfolios the student marks as public. Any unauthenticated user possessing the link shall be able to view the portfolio in read-only mode. |
| **FR-S-8** | The system shall display all in-app notifications for the authenticated student, ordered by most recent first, with unread count indicated. |

| | |
|---|---|
| **FR-S-9** | The system shall allow a student to mark individual notifications as read or mark all as read with a single action; the unread count shall update immediately in the UI. |

*4.1.3 Flow Description*

1. Student navigates to /login and submits credentials.
2. On success, a JWT is issued and stored in localStorage; user is redirected to /dashboard.
3. Dashboard fetches and renders achievement stats and recent notifications via authenticated API calls.
4. Student navigates to /achievements to manage submissions.
5. Student navigates to /portfolio to manage portfolios and toggle public sharing.

*4.1.4 Error Handling*

| ***Scenario*** | ***Expected Behaviour*** |
|---|---|
| Duplicate achievement title for the same student | Return HTTP 409; display "An achievement with this title already exists." |
| Missing required fields on submission | Return HTTP 400; highlight each invalid field inline |
| Expired JWT token | Return HTTP 401; redirect client to /login with a session-expired toast message |
| Portfolio public toggle when shareLink generation fails | Return HTTP 500; notify student to retry; portfolio visibility remains unchanged |

## 4.2 *Faculty (Teacher) Dashboard*

*4.2.1 Feature Description*

The Faculty Dashboard provides faculty members with a queue of student achievements awaiting review, tools to record evaluations, and a historical view of all reviews they have performed. Access is restricted to users with the faculty role.

*4.2.2 Functional Requirements*

| ID | Requirement |
|---|---|
| **FR-T-1** | The system shall display to the faculty member a list of all achievements with status pending or under_review across all students, sorted by submission date (oldest first). |
| **FR-T-2** | The system shall allow a faculty member to open an achievement's detail view and read all submitted information including student name, title, description, category, skills, and supporting document URLs. |

| | |
|---|---|
| **FR-T-3** | The system shall allow a faculty member to submit a review decision (approved, rejected, or needs_info) accompanied by a mandatory comment (min 10 characters) and a numeric rating (0.0 to 5.0). |
| **FR-T-4** | Upon review submission, the system shall atomically update the achievement's status to match the review decision, record the reviewer's user ID, and persist the review record in the reviews collection. |
| **FR-T-5** | Upon review submission, the system shall automatically create and dispatch an in-app notification to the student indicating the achievement title and the review outcome. |
| ID | Requirement |
| **FR-T-6** | The system shall prevent a faculty member from submitting a review for the same achievement more than once; a second submission attempt shall return HTTP 409. |
| **FR-T-7** | The system shall allow a faculty member to view the complete history of all reviews they have previously submitted, including the achievement title, student name, rating, and comment. |

*4.2.3 Flow Description*

1. Faculty logs in; redirected to /dashboard populated with pending review count and recent activity.
2. Faculty navigates to /reviews.
3. System presents paginated list of reviewable achievements.
4. Faculty opens a specific achievement, reads evidence, then submits a review form (decision + rating + comment).
5. System persists review, updates achievement status, dispatches notification to student.

*4.2.4 Error Handling*

| ***Scenario*** | ***Expected Behaviour*** |
|---|---|
| Empty comment on review submission | Return HTTP 400; display "Review comment is required." |
| Rating outside 0–5 range | Return HTTP 400; display "Rating must be between 0 and 5." |
| Achievement already reviewed | Return HTTP 409; display "This achievement has already been reviewed." |
| Achievement not found | Return HTTP 404; display "Achievement does not exist or has been removed." |

## 4.3 *Admin Dashboard*

*4.3.1 Feature Description*

The Admin Dashboard provides system administrators with full control over user accounts, system configuration parameters, and the immutable audit log. The admin role carries the highest privilege level in SPAMS.

*4.3.2 Functional Requirements*

| ID | Requirement |
|---|---|
| **FR-A-1** | The system shall display to the admin a summary dashboard showing: total users, users by role, recently created accounts, and any system health flags. |
| ID | Requirement |
| **FR-A-2** | The system shall allow the admin to retrieve a paginated list of all registered users, filterable by role (student, faculty, department_head, admin) and status (active, inactive). |
| **FR-A-3** | The system shall allow the admin to create a new user account by supplying: name, email (unique), password (minimum 8 characters), role, and department. |
| **FR-A-4** | The system shall allow the admin to update any user's profile fields (name, department, role, status) without resetting the user's password. |
| **FR-A-5** | The system shall allow the admin to deactivate (set status = 'inactive') a user account; deactivated users shall receive HTTP 401 on subsequent login attempts. |
| **FR-A-6** | The system shall allow the admin to permanently delete a user account; the system shall cascade-nullify or retain associated achievement and portfolio records per configured retention policy. |
| **FR-A-7** | The system shall allow the admin to modify system configuration values (e.g., list of achievement categories, system name, academic year), which are persisted in the system_configs collection. |
| **FR-A-8** | The system shall display the full audit log, with entries for: actor (user ID + name + role), action performed, target entity (type + ID), timestamp, and IP address. |
| **FR-A-9** | The system shall provide filtering of the audit log by actor, action type, and date range. |
| **FR-A-10** | The system shall prevent any user from modifying the audit log; all audit entries are created system-internally and are immutable via the API. |

*4.3.3 Flow Description*

1. Admin logs in; redirected to /dashboard with system overview.

2. Admin navigates to /users for user management operations.
3. Admin navigates to /config to manage system settings.
4. Admin navigates to /audit for a read-only view of the audit log.

*4.3.4 Error Handling*

| ***Scenario*** | ***Expected Behaviour*** |
|---|---|
| Duplicate email on user creation | Return HTTP 409; display "A user with this email already exists." |
| Attempt to delete own admin account | Return HTTP 403; display "Administrators cannot delete their own account via this interface." |
| Invalid role value on user update | Return HTTP 400; display "Invalid role specified." |
| Empty config key on configuration update | Return HTTP 400; display "Configuration key is required." |

## 4.4 *Department Head Dashboard*

*4.4.1 Feature Description*

The Department Head Dashboard provides senior academic staff with a read-only, high-level analytics view of achievement and portfolio activity across the department. Department Heads do not manage individual user accounts but consume aggregated reports to support informed academic decision-making.

*4.4.2 Functional Requirements*

| ***ID*** | ***Requirement*** |
|---|---|
| **FR-D-1** | The system shall display to the Department Head a summary of total achievements submitted, broken down by status (pending, under_review, approved, rejected). |
| **FR-D-2** | The system shall render a bar or pie chart displaying the distribution of approved achievements by category. |
| **FR-D-3** | The system shall display a time-series chart showing the volume of achievement submissions over time (weekly or monthly granularity). |
| **FR-D-4** | The system shall display a leaderboard or ranked list of students ordered by their total number of approved achievements. |
| **FR-D-5** | The system shall display the average rating assigned to approved achievements across the department. |
| **FR-D-6** | The system shall allow the Department Head to filter all analytics views by department (if multiple departments exist) and by date range. |

| | |
|---|---|
| **FR-D-7** | The system shall not expose any individual student PII (e.g., passwords, contact details) in the analytics view; only name and academic performance metrics are permissible. |

*4.4.3 Flow Description*

1. Department Head logs in; redirected to /dashboard with top-level KPIs.
2. Department Head navigates to /analytics to access full charted reports.
3. Charts render from pre-computed aggregate data returned by the backend analytics endpoints.
4. Department Head applies date range or department filters; charts update dynamically.

*4.4.4 Error Handling*

| ***Scenario*** | ***Expected Behaviour*** |
|---|---|
| No achievements exist for selected filter | Display "No data available for the selected period" within the chart area. |
| Analytics API timeout | Display a retry prompt with "Unable to load data, please try again." |
| Attempt to access /users or /config routes | Redirect to /dashboard with HTTP 403 toast; navigation items for these routes are not rendered for this role. |

# 5. Functional Requirements (Master List)

All requirements are uniquely numbered, role-tagged, and written to be independently testable.

| ***ID*** | ***Actor*** | ***Requirement Statement*** |
|---|---|---|
| **FR-1** | All | The system shall authenticate users via email and password, issuing a signed JWT upon successful credential verification. |
| **FR-2** | All | The system shall reject authentication requests with incorrect credentials with HTTP 401 and the message "Invalid email or password." |
| **FR-3** | All | The system shall expire JWT tokens after 7 days; expired tokens shall result in HTTP 401 on any protected endpoint. |
| **FR-4** | All | The system shall enforce role-based access control on every API endpoint; a request from a role not authorised for that endpoint shall return HTTP 403. |
| **FR-5** | All | The system shall display only the navigation items applicable to the authenticated user's role. |
| **FR-6** | All | The system shall provide a logout function that removes the JWT from localStorage and redirects the user to /login. |

| | | |
|---|---|---|
| **FR-7** | All | The system shall deliver in-app notifications to users on relevant system events. |
| **FR-8** | All | The system shall allow any authenticated user to mark individual notifications as read via HTTP PUT. |
| **FR-9** | All | The system shall allow any authenticated user to mark all their notifications as read in a single operation. |
| **FR-10** | Student | The system shall allow students to submit achievements with the fields defined in FR-S-2. |
| **FR-11** | Student | The system shall validate achievement submission fields serverside and return descriptive per-field errors on failure. |
| **FR-12** | Student | The system shall allow students to view all their previously submitted achievements and their current review status. |
| **FR-13** | Student | The system shall allow students to view the review comment, reviewer name, and rating for each reviewed achievement. |
| **FR-14** | Student | The system shall allow students to create, update, and delete portfolio records. |
| **FR-15** | Student | The system shall allow students to add portfolio items (title, description, duration, attachments) to their portfolios. |
| **FR-16** | Student | The system shall generate a unique shareLink for portfolios set to public visibility. |
| **FR-17** | Student | The system shall allow unauthenticated read-only access to public portfolios via their shareLink. |

| ***ID*** | ***Actor*** | ***Requirement Statement*** |
|---|---|---|
| **FR-18** | Faculty | The system shall present faculty members with a queue of achievements in pending or under_review status. |
| **FR-19** | Faculty | The system shall allow faculty members to view the full detail of any achievement in the review queue. |
| **FR-20** | Faculty | The system shall allow faculty members to submit a review comprising a decision, mandatory comment, and rating. |
| **FR-21** | Faculty | The system shall update the achievement's status atomically upon review submission. |
| **FR-22** | Faculty | The system shall prevent a faculty member from submitting duplicate reviews for the same achievement. |

| | | |
|---|---|---|
| **FR-23** | Faculty | The system shall dispatch a notification to the student upon faculty review submission. |
| **FR-24** | Faculty | The system shall provide faculty members with a history view of all reviews they have submitted. |
| **FR-25** | Admin | The system shall allow admins to create, read, update, and deactivate user accounts for any role. |
| **FR-26** | Admin | The system shall enforce unique email constraint on user creation; duplicate emails shall return HTTP 409. |
| **FR-27** | Admin | The system shall prevent login for users with status = 'inactive'. |
| **FR-28** | Admin | The system shall allow admins to permanently delete user accounts. |
| **FR-29** | Admin | The system shall allow admins to modify system configuration values. |
| **FR-30** | Admin | The system shall record all configuration changes in the audit log. |
| **FR-31** | Admin | The system shall display the full audit log to admins, filterable by actor, action, and date. |
| **FR-32** | Admin | The system shall prohibit any API route from modifying audit log entries once created. |
| **FR-33** | Dept. Head | The system shall display a summary of total achievements by status on the Department Head dashboard. |
| **FR-34** | Dept. Head | The system shall render a category distribution chart for approved achievements. |
| **FR-35** | Dept. Head | The system shall render a time-series submission volume chart. |
| **FR-36** | Dept. Head | The system shall display a student leaderboard ordered by approved achievement count. |
| **FR-37** | Dept. Head | The system shall display the department-wide average achievement rating. |
| **FR-38** | Dept. Head | The system shall support date range and department-level |
| ***ID*** | ***Actor*** | ***Requirement Statement*** |
| | | filtering on all analytics views. |

# 6. **Non-Functional Requirements**

## 6.1 *Performance*

| *ID* | *Requirement* |
|---|---|
| NFR-P-1 | The system shall render the initial dashboard page (after login redirect) within 2 seconds on a standard broadband connection (≥ 10 Mbps). |
| NFR-P-2 | All REST API responses for non-aggregate reads (single resource or paginated list up to 50 items) shall be returned within 500 milliseconds at the 95th percentile under normal load (≤ 50 concurrent users). |
| NFR-P-3 | Analytics aggregation endpoints shall return results within 3 seconds for datasets of up to 10,000 achievement records. |
| NFR-P-4 | The system shall support up to 200 concurrent authenticated users without response time degradation exceeding 20% of baseline. |

### 6.2 *Security*

| *ID* | *Requirement* |
|---|---|
| NFR-S-1 | All passwords stored in the database shall be hashed using bcrypt with a minimum cost factor of 10. Plain-text passwords shall never be persisted or returned via any API response. |
| NFR-S-2 | All JWT tokens shall be signed using HS256 with a secret of at least 256 bits of entropy. Token payloads shall include `sub` (user ID), `role`, and `exp` (expiry) claims. |
| NFR-S-3 | The system shall enforce RBAC on every protected endpoint via a NestJS Guard; unauthorised role access shall return HTTP 403. |
| NFR-S-4 | The backend shall validate and sanitise all incoming request bodies using `class-validator` DTOs; malformed or unexpected fields shall be stripped. |
| NFR-S-5 | The system shall enforce CORS allowing requests only from the configured frontend origin. |
| NFR-S-6 | The system shall not expose any password, hashed or plain, in API responses. The `toJSON` transform on the `User` schema shall explicitly delete the password field on serialisation. |
| NFR-S-7 | The system shall be protected against the OWASP Top 10 vulnerability classes, specifically: SQL/NoSQL injection, broken authentication, security misconfiguration, and sensitive data exposure. |

### 6.3 *Availability*

| *ID* | *Requirement* |
|---|---|
| NFR-AV-1 | The system shall maintain an uptime of at least 99% within any 30-day rolling window during the academic testing period. |

| | |
|---|---|
| NFR-AV-2 | The system shall implement graceful error handling; any unhandled exception on the backend shall return HTTP 500 with a user-safe error message and shall not expose internal stack traces to the client. |

### 6.4 *Scalability*

| ***ID*** | ***Requirement*** |
|---|---|
| NFR-SC-1 | The database schema shall support at least 5,000 user accounts and 50,000 achievement records without schema migration. |
| NFR-SC-2 | MongoDB indexes shall be defined on high-frequency query fields: achievements.studentId, achievements.status, reviews.achievementId, notifications.userId. |
| NFR-SC-3 | The backend shall be stateless and horizontally scalable; no session state shall be stored in-process. |

### 6.5 *Maintainability*

| ***ID*** | ***Requirement*** |
|---|---|
| NFR-M-1 | The backend codebase shall adhere to the NestJS modular architecture pattern; each domain entity shall have its own dedicated module. |
| NFR-M-2 | All API endpoints shall follow RESTful naming conventions (plural nouns, HTTP verbs maps to CRUD semantics). |
| NFR-M-3 | The codebase shall maintain a minimum of 70% line coverage through unit and integration tests over all service-layer classes. |
| NFR-M-4 | All environment-dependent configuration (database URI, JWT secret, port number) shall be managed via .env files and shall not be hard-coded in source files. |

### 6.6 *Usability*

| ***ID*** | ***Requirement*** |
|---|---|
| NFR-U-1 | The UI shall be responsive and shall render correctly on screen widths from 375px (mobile) to 2560px (4K desktop). |
| NFR-U-2 | The system shall display meaningful, non-technical error messages for all user-facing error states. |
| NFR-U-3 | All interactive navigation elements shall provide visual feedback (hover state, active state) within 100ms of user interaction. |
| NFR-U-4 | The system shall display a loading state whenever an asynchronous API |
| ***ID*** | ***Requirement*** |
| | call is in flight, preventing user interaction with stale data. |

| | |
|---|---|
| NFR-U-5 | All form inputs shall provide inline validation feedback within 300ms of the user leaving the field (onBlur validation). |

### 6.7 *Audit & Logging*

| ***ID*** | ***Requirement*** |
|---|---|
| NFR-AL-1 | The system shall record an audit log entry for every mutating operation (POST, PUT, PATCH, DELETE) performed on protected resources, capturing: actor user ID, actor name, actor role, action descriptor, target entity type, target entity ID, timestamp (UTC), and client IP address. |
| NFR-AL-2 | Audit log entries shall be immutable; no API endpoint shall expose update or delete operations on audit log records. |
| NFR-AL-3 | The system shall retain audit logs for a minimum of 12 months. |
| NFR-AL-4 | The audit log shall be queryable by admin users with filters for date range (specific to day precision), actor, and action type. |